SECURITY AUDIT REPORT

RasPWN

OBJECT	EDITOR	DATE
RasPWN security report	Ian Hill	2026-10-12

VERSION	DATE	EDITOR'S NAME	OBJECT
1.0	2026-10-10	Ian Hill	Document creation

SUMMARY

1. CONTACTS....2
2. INTRODUCTION....4
2.1 Mailing list....4
2.2 Period and Confidentiality....4
2.3 Perimeter....4
3. EXECUTIVE SUMMARY....6
3.1 Presentation of vulnerabilities and findings....7
3.2 Presentation of recommendations....9
4. DETAILED WORKS....11
4.1 Vulnerabilities research....11
4.2 Discovered VulnerabilitIes....12
5. ANNEXES....14
5.1 Figures table....14
5.2 Vulnerabilities classification – CVSS Scorring....15
5.3 Top 10 OWASP 2021....16

1. CONTACTS

NAME	OCCUPATION	PHONE	E-MAIL
Ian Hill	Cybersecurity consultant	undefined	undefined

CONTEXT, OBJECTIVES AND SCOPE

2. INTRODUCTION

2.1 MAILING LIST

COMPANY	PERSON	OCCUPATION
RasPWN	Raspberry Pwnd	PwnMaster
Hitsaru, LLC	Ian Hill	Cybersecurity consultant

2.2 PERIOD AND CONFIDENTIALITY

The security audit took place over the following period(s):

AUDIT ACTIVITY	DATE
Penetration Testing	From 2026-09-01 to 2026-10-10

All data collected during the audit will be transmitted to their owner (RasPWN) upon request and/or destroyed at the end of the mission.

2.3 PERIMETER

2.3.1 Technical perimeter

RasPWN notified [HOST/PROVIDER WOULD GO HERE] of the Penetration Test authorization and provided the following resources:

- 192.168.99.1
- 192.168.99.13
- 192.168.99.7
- 192.168.99.10

The entire service is performed remotely from public IP addresses: [PUBLIC Ips WOULD GO HERE].

EXECUTIVE SUMMARY

3. EXECUTIVE SUMMARY

This is not a real report. This document is intended as a sample report for the Faraday CE to PWNDOC-NG report generator at

https://github.com/hitsaru-i/faradayceToPwndocng-ReportGenerator

This script is intended to allow users to create custom reports like these from aggregate vulnerability scans and workflows management in Faraday CE and docx based reports generated from templates with PWNDOC-NG. The script facilitates piping outputs from FaradayCE API to the PWNDOC-NG API and transforming the relevant data points for expeditious report generation.

The sample target system is a Raspberry Pi running RasPWN OS, an intentionally vulnerably pentesting lab hosting it's own services. Only a few of the MANY vulnerabilities are provided here. Formatting and quality/quantity of vulnerability reporting is not the point in this document.

Most of the output in this document is automatically generated content from vulnerability scanners.

3.1 PRESENTATION OF VULNERABILITIES AND FINDINGS

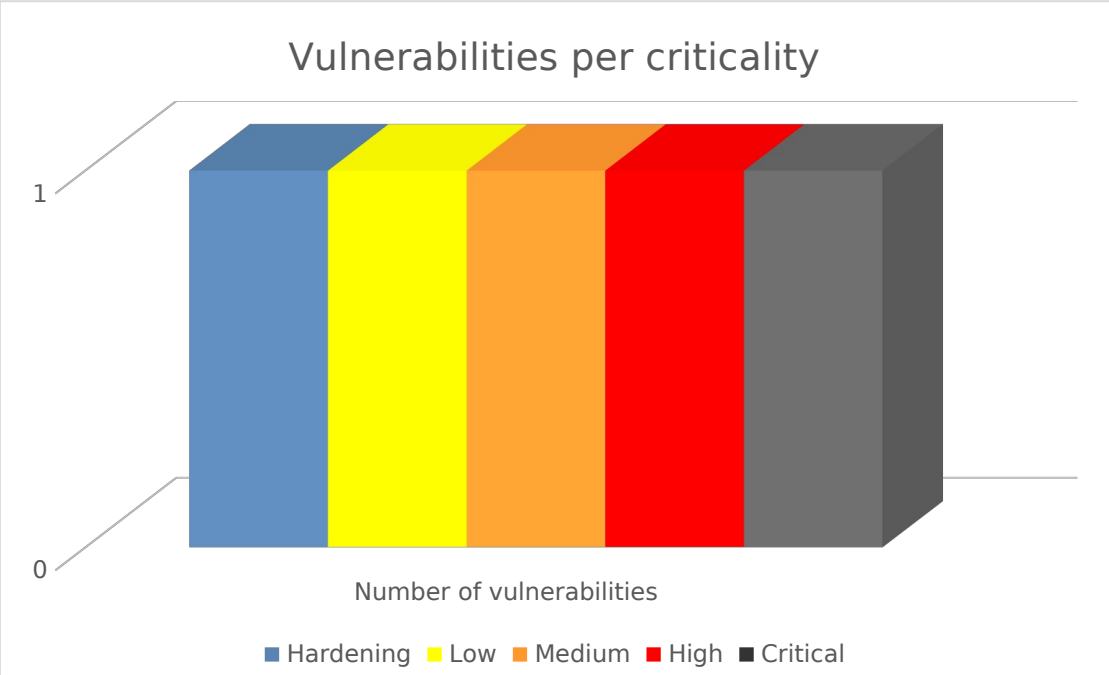
The table below lists the vulnerabilities observed during the audit. Each vulnerability is associated with one or more recommendations, with one or more threats and with a severity level based on the CVSS scale, i.e. according to the impact and the ease of exploitation, in accordance state of the art.

REFERENCE	TITLE	SCOPE	CVSS SCORE	RISK(S)
	Operating System (OS) End of Life (EOL) Detection	192.168.99.13	CVSS 10.0	The Operating System (OS) on the remote host has reached the end of life (EOL) and should not be used anymore.
	Operating System (OS) End of Life (EOL) Detection	192.168.99.10	CVSS 10.0	The Operating System (OS) on the remote host has reached the end of life (EOL) and should not be used anymore.
	Operating System (OS) End of Life (EOL) Detection	192.168.99.7	CVSS 10.0	The Operating System (OS) on the remote host has reached the end of life (EOL) and should not be used anymore.
	SQL Injection - MySQL	192.168.99.13	CVSS 8.8	SQL injection is possible.
	Remote OS Command Injection	192.168.99.13	CVSS 8.8	Attack technique used for unauthorized execution of operating system commands. This attack is possible when an application accepts untrusted input to build operating system commands in an insecure manner involving

				improper data sanitization, and/or improper calling of external programs.
	Path Traversal	192.168.99.13	CVSS 6.5	The Path Traversal attack technique allows an attacker access to files, directories, and commands that potentially reside outside the web document root directory. An attacker may manipulate a URL in such a way that the web site will execute or reveal the contents of arbitrary files anywhere on the web server. Any device that exposes an HTTP-based interface is potentially vulnerable to Path Traversal. Most web sites restrict user access to a specific portion of the file-system, typically called the "web document root" or "CGI root" directory. These directories contain the files intended for user access and the executable necessary to drive web application functionality. To access files or execute commands anywhere on the file-system, Path Traversal attacks will utilize the ability of special-characters sequences. The most basic Path Traversal attack uses the "../" special-character sequence to alter the resource location requested in the URL. Although most popular web servers will prevent this technique from escaping the web document root, alternate encodings of the "../" sequence may help bypass the security filters. These method variations include valid and invalid Unicode-encoding ("..%u2216" or "..%c0%af") of the forward slash character, backslash characters ("..\") on Windows-based servers, URL encoded characters "%2e%2e%2f"), and double URL encoding ("..%255c") of the backslash character. Even if the web server properly restricts Path Traversal attempts in the URL path, a web application itself may still be vulnerable due to improper handling of user-supplied input. This is a common problem of web applications that use template mechanisms or load static text from

				files. In variations of the attack, the original URL parameter value is substituted with the file name of one of the web application's dynamic scripts. Consequently, the results can reveal source code because the file is interpreted as text instead of an executable script. These techniques often employ additional special characters such as the dot (".") to reveal the listing of the current working directory, or "%00" NULL characters in order to bypass rudimentary file extension checks.
	Cleartext Transmission of Sensitive Information via HTTP	192.168.99.1	CVSS 5.4	The host / application transmits sensitive information (username, passwords) in cleartext via HTTP.
	phpinfo() Output Reporting (HTTP)	192.168.99.13	CVSS 5.3	Reporting of files containing the output of the phpinfo() PHP function previously detected via HTTP. Many PHP installation tutorials instruct the user to create a file called phpinfo.php or similar containing the phpinfo() statement. Such a file is often left back in the webserver directory. http://playground.raspwn.org/mutillidae/phpinfo.php
	SSL/TLS: Certificate Expired	192.168.99.13	CVSS 5.3	The remote server's SSL/TLS certificate has already expired. This script checks expiry dates of certificates associated with SSL/TLS-enabled services on the target and reports whether any have already expired.

The graph below provides an overall view of the level and number of vulnerabilities in the scope audited. The presence of 3 vulnerabilities qualified as « critical », of 2 vulnerabilities qualified as « high » and of 4 vulnerabilities qualified as « medium »



3.2 PRESENTATION OF RECOMMENDATIONS

The table below lists the recommendations for dealing with vulnerabilities or audit findings.

Each finding or vulnerability is associated with one or more risks, one or more recommendations and a level of priority.

REFERENCE	TITLE	SCOPE	RECOMMENDATION(S)	COMPLEXITY	PRIORITY LEVEL
	Operating System (OS) End of Life (EOL) Detection	192.168.99.13	Update the OS on the remote host to a version which is still supported and receiving security updates by the vendor. Note / Important: Please create an override for this result if the target host is a: - Windows system with Extended Security Updates (ESU) - System with additional 3rd-party / non-vendor security updates like e.g. from 'TuxCare', 'Freexian Extended LTS' or similar	High	4
	Operating System (OS) End of Life (EOL) Detection	192.168.99.10	Update the OS on the remote host to a version which is still supported and receiving security updates by the vendor. Note / Important: Please create an override for this result if the target host is a: - Windows system with Extended Security Updates (ESU) - System with additional 3rd-party / non-vendor security updates like e.g. from 'TuxCare', 'Freexian Extended LTS' or similar	High	4
	Operating System (OS) End of Life (EOL) Detection	192.168.99.7	Update the OS on the remote host to a version which is still supported and receiving security updates by the vendor. Note / Important: Please create an override for this result if the target host is a: - Windows system with Extended Security Updates (ESU) - System with additional 3rd-party / non-vendor security updates like e.g. from 'TuxCare', 'Freexian Extended LTS' or similar	High	4

	SQL Injection - MySQL	192.168.99.13	Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?' If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries. If database Stored Procedures can be used, use them. Do *not* concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality! Do not create dynamic SQL queries using simple string concatenation. Escape all data received from the client. Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input. Apply the principle of least privilege by using the least privileged database user possible. In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact. Grant the minimum database access that is necessary for the application.	High	3
	Remote OS Command Injection	192.168.99.13	If at all possible, use library calls rather than external processes to recreate the desired functionality. Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general,	High	3

			managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise. For any data that will be used to generate a command to be executed, keep as much of that data out of external control as possible. For example, in web applications, this may require storing the command locally in the session's state instead of sending it out to the client in a hidden form field. Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, consider using the ESAPI Encoding control or a similar tool, library, or framework. These will help the programmer encode outputs in a manner less prone to error. If you need to use dynamically-generated query strings or commands in spite of the risk, properly quote arguments and escape any special characters within those arguments. The most conservative approach is to escape or filter all characters that do not pass an extremely strict allow list (such as everything that is not alphanumeric or white space). If some special characters are still needed, such as white space, wrap each argument in quotes after the escaping/filtering step. Be careful of argument injection. If the program to be executed allows arguments to be specified within an input file		

			or from standard input, then consider using that mode to pass arguments instead of the command line. If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated. Some languages offer multiple functions that can be used to invoke commands. Where possible, identify any function that invokes a command shell using a single string, and replace it with a function that requires individual arguments. These functions typically perform appropriate quoting and filtering of arguments. For example, in C, the system() function accepts a string that contains the entire command to be executed, whereas execl(), execve(), and others require an array of strings, one for each argument. In Windows, CreateProcess() only accepts one command at a time. In Perl, if system() is provided with an array of arguments, then it will quote each of the arguments. Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider		

			all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." When constructing OS command strings, use stringent allow lists that limit the character set based on the expected value of the parameter in the request. This will indirectly limit the scope of an attack, but this technique is less important than proper output encoding and escaping. Note that proper output encoding, escaping, and quoting is the most effective solution for preventing OS command injection, although input validation may provide some defense-in-depth. This is because it effectively limits what will appear in output. Input validation will not always prevent OS command injection, especially if you are required to support free-form text fields that could contain arbitrary characters. For example, when invoking a mail program, you might need to allow the subject field to contain otherwise-dangerous inputs like ";" and ">" characters, which would need to be escaped or otherwise handled. In this case, stripping the character might reduce the risk of OS command injection, but it would produce incorrect behavior because the subject field would not be recorded as the user intended. This might seem to be a minor inconvenience, but it could be more important when the program relies on well-structured subject lines in order to pass messages to other components.		

			Even if you make a mistake in your validation (such as forgetting one out of 100 input fields), appropriate encoding is still likely to protect you from injection-based attacks. As long as it is not done in isolation, input validation is still a useful technique, since it may significantly reduce your attack surface, allow you to detect some attacks, and provide other security benefits that proper encoding does not address.		
	Path Traversal	192.168.99.13	Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." For filenames, use stringent allow lists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses, and exclude directory separators such as "/". Use an allow list of allowable file extensions.	Medium	3

			Warning: if you attempt to cleanse your data, then do so that the end result is not in the form that can be dangerous. A sanitizing mechanism can remove characters such as '.' and ';' which may be required for some exploits. An attacker can try to fool the sanitizing mechanism into "cleaning" data into a dangerous form. Suppose the attacker injects a '.' inside a filename (e.g. "sensi.tiveFile") and the sanitizing mechanism removes the character resulting in the valid filename, "sensitiveFile". If the input data are now assumed to be safe, then the file may be compromised. Inputs should be decoded and canonicalized to the application's current internal representation before being validated. Make sure that your application does not decode the same input twice. Such errors could be used to bypass allow list schemes by introducing dangerous inputs after they have been checked. Use a built-in path canonicalization function (such as realpath() in C) that produces the canonical version of the pathname, which effectively removes ".." sequences and symbolic links. Run your code using the lowest privileges that are required to accomplish the necessary tasks. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations. When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input		

			values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs. Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise.		
	Cleartext Transmission of Sensitive Information via HTTP	192.168.99.1	Enforce the transmission of sensitive data via an encrypted SSL/TLS connection. Additionally make sure the host / application is redirecting all users to the secured SSL/TLS connection before allowing to input sensitive data into the mentioned functions.	Medium	2
	phpinfo() Output Reporting (HTTP)	192.168.99.13	Delete the listed files or restrict access to them.	Low	2
	SSL/TLS: Certificate	192.168.99.13	Replace the SSL/TLS certificate by a new one.	Low	2

	Expired				

DETAILED WORK

4. DETAILED WORKS

4.1 VULNERABILITIES RESEARCH

I pointed some scanners at some services and cherry picked a few of the easy ones to report.

Almost all of the content is automatically generated by various scanners and parsed into this format with the faradayceToPwndocng-ReportGenerator script.

4.2 DISCOVERED VULNERABILITIES

4.2.1 Operating System (OS) End of Life (EOL) Detection

		CVSS 10.0
Title	**Operating System (OS) End of Life (EOL) Detection**	
Scope	192.168.99.13	
Description	The Operating System (OS) on the remote host has reached the end of life (EOL) and should not be used anymore.	
Observation	Target: 192.168.99.13 Hostname(s): playground.raspwn.org, 192.168.99.13 Operating System: Linux Tool: Openvas External ID: OPENVAS-1.3.6.1.4.1.25623.1.0.103674	
POC		
Recommendation	Update the OS on the remote host to a version which is still supported and receiving security updates by the vendor. Note / Important: Please create an override for this result if the target host is a: - Windows system with Extended Security Updates (ESU) - System with additional 3rd-party / non-vendor security updates like e.g. from 'TuxCare', 'Freexian Extended LTS' or similar	
References	cpe:/o:debian:debian_linux:7 SEVERITY NUMBER: 10.0 THREAT: Critical	

4.2.2 Operating System (OS) End of Life (EOL) Detection

		CVSS 10.0
Title	**Operating System (OS) End of Life (EOL) Detection**	
Scope	192.168.99.10	
Description	The Operating System (OS) on the remote host has reached the end of life (EOL) and should not be used anymore.	
Observation	Target: 192.168.99.10 Hostname(s): ns2.playground.raspwn.org, wackopicko.playground.raspwn.org Operating System: Linux Tool: Openvas External ID: OPENVAS-1.3.6.1.4.1.25623.1.0.103674	
POC		
Recommendation	Update the OS on the remote host to a version which is still supported and receiving security updates by the vendor. Note / Important: Please create an override for this result if the target host is a: - Windows system with Extended Security Updates (ESU) - System with additional 3rd-party / non-vendor security updates like e.g. from 'TuxCare', 'Freexian Extended LTS' or similar	
References	cpe:/o:debian:debian_linux:7 SEVERITY NUMBER: 10.0 THREAT: Critical	

4.2.3 Operating System (OS) End of Life (EOL) Detection

		CVSS 10.0
Title	**Operating System (OS) End of Life (EOL) Detection**	
Scope	192.168.99.7	
Description	The Operating System (OS) on the remote host has reached the end of life (EOL) and should not be used anymore.	
Observation	Target: 192.168.99.7 Hostname(s): nginx.playground.raspwn.org Operating System: Linux Tool: Openvas External ID: OPENVAS-1.3.6.1.4.1.25623.1.0.103674	
POC		
Recommendation	Update the OS on the remote host to a version which is still supported and receiving security updates by the vendor. Note / Important: Please create an override for this result if the target host is a: - Windows system with Extended Security Updates (ESU) - System with additional 3rd-party / non-vendor security updates like e.g. from 'TuxCare', 'Freexian Extended LTS' or similar	
References	cpe:/o:debian:debian_linux:7 SEVERITY NUMBER: 10.0 THREAT: Critical	

4.2.4 SQL Injection - MySQL

		CVSS 8.8
Title	**SQL Injection - MySQL**	
Scope	192.168.99.13	
Description	SQL injection is possible.	
Observation	Target: 192.168.99.13 Hostname(s): playground.raspwn.org, 192.168.99.13 Operating System: Linux Tool: Zap External ID: ZAP-40018 Web Vulnerability Findings: Website: http://192.168.99.13 Path: /dvwa/vulnerabilities/sqli/ Parameters: id, Submit Parameter name: id Query: id=%27&Submit=Submit Method: GET	

POC	**HTTP Request**
	`GET http://192.168.99.13/dvwa/vulnerabilities/sqli/?id=%27&Submit=Submit HTTP/1.1`
	`host: 192.168.99.13`
	`User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0`
	`Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8`
	`Accept-Language: en-US,en;q=0.5`
	`Connection: keep-alive`
	`Referer: https://192.168.99.13/dvwa/vulnerabilities/sqli/?id=2&Submit=Submit`
	`Cookie: security=low; PHPSESSID=e4b816vebcgoufdmiibn9bcse2`
	`Upgrade-Insecure-Requests: 1`
	`Sec-Fetch-Dest: document`
	`Sec-Fetch-Mode: navigate`
	`Sec-Fetch-Site: same-origin`
	`Sec-Fetch-User: ?1`
	`Priority: u=0, i`
	HTTP Response
	`HTTP/1.1 200 OK`
	`Date: Tue, 06 Sep 2016 14:03:07 GMT`
	`Server: Apache/2.2.22 (Debian)`
	`X-Powered-By: PHP/5.4.36-0+deb7u1`
	`Expires: Thu, 19 Nov 1981 08:52:00 GMT`
	`Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0`
	`Pragma: no-cache`
	`Vary: Accept-Encoding`
	`Content-Length: 160`
	`Keep-Alive: timeout=5, max=85`	

	Connection: Keep-Alive Content-Type: text/html <pre>You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''''' at line 1</pre>
Recommendation	Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?' If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries. If database Stored Procedures can be used, use them. Do *not* concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality! Do not create dynamic SQL queries using simple string concatenation. Escape all data received from the client. Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input. Apply the principle of least privilege by using the least privileged database user possible. In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact. Grant the minimum database access that is necessary for the application.
References	CWE: CWE-89 https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html WASC:19

4.2.5 Remote OS Command Injection

		CVSS 8.8
Title	**Remote OS Command Injection**	
Scope	192.168.99.13	
Description	Attack technique used for unauthorized execution of operating system commands. This attack is possible when an application accepts untrusted input to build operating system commands in an insecure manner involving improper data sanitization, and/or improper calling of external programs.	
Observation	Target: 192.168.99.13 Hostname(s): playground.raspwn.org, 192.168.99.13 Operating System: Linux Tool: Zap External ID: ZAP-90020 Web Vulnerability Findings: Website: https://192.168.99.13 Path: /dvwa/vulnerabilities/exec/ Parameter name: ip Method: POST	

POC	**HTTP Request**
	`POST https://192.168.99.13/dvwa/vulnerabilities/exec/ HTTP/1.1`
	`host: 192.168.99.13`
	`User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0`
	`Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8`
	`Accept-Language: en-US,en;q=0.5`
	`Content-Type: application/x-www-form-urlencoded`
	`content-length: 51`
	`Origin: https://192.168.99.13`
	`Connection: keep-alive`
	`Referer: https://192.168.99.13/dvwa/vulnerabilities/exec/`
	`Cookie: security=low; PHPSESSID=e4b816vebcgoufdmiibn9bcse2`
	`Upgrade-Insecure-Requests: 1`
	`Sec-Fetch-Dest: document`
	`Sec-Fetch-Mode: navigate`
	`Sec-Fetch-Site: same-origin`
	`Sec-Fetch-User: ?1`
	`Priority: u=0, i`
	
	`ip=127.0.0.1%26cat+%2Fetc%2Fpasswd%26&Submit=Submit`
	HTTP Response
	`HTTP/1.1 200 OK`
	`Date: Tue, 06 Sep 2016 13:14:28 GMT`
	`Server: Apache/2.2.22 (Debian)`
	`X-Powered-By: PHP/5.4.36-0+deb7u1`
	`Expires: Tue, 23 Jun 2009 12:00:00 GMT`
	`Cache-Control: no-cache, must-revalidate`	

```
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 6834
Keep-Alive: timeout=5, max=99
Connection: Keep-Alive
Content-Type: text/html;charset=utf-8


<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

	<head>
		<meta http-equiv="Content-Type" content="text/html;
charset=UTF-8" />

		<title>Vulnerability: Command Injection :: Damn
Vulnerable Web Application (DVWA) v1.9</title>

		<link rel="stylesheet" type="text/css"
href="../../dvwa/css/main.css" />

		<link rel="icon" type="\image/ico"
href="../../favicon.ico" />

		<script type="text/javascript"
src="../../dvwa/js/dvwaPage.js"></script>

	</head>

	<body class="home">
		<div id="container">

			<div id="header">

				<img src="../../dvwa/images/logo.png"
alt="Damn Vulnerable Web Application" />

			</div>

			<div id="main_menu">

				<div id="main_menu_padded">
				<ul class="menuBlocks"><li
onclick="window.location='../../.'" class=""><a
href="../../.">Home</a></li>
<li onclick="window.location='../../instructions.php'" class=""><a
href="../../instructions.php">Instructions</a></li>
<li onclick="window.location='../../setup.php'" class=""><a
href="../../setup.php">Setup / Reset DB</a></li>
</ul><ul class="menuBlocks"><li
onclick="window.location='../../vulnerabilities/brute/'" class=""><a
href="../../vulnerabilities/brute/">Brute Force</a></li>
<li onclick="window.location='../../vulnerabilities/exec/'"
class="selected"><a href="../../vulnerabilities/exec/">Command
Injection</a></li>
<li onclick="window.location='../../vulnerabilities/csrf/'" class=""><a
href="../../vulnerabilities/csrf/">CSRF</a></li>
<li onclick="window.location='../../vulnerabilities/fi/.?
page=include.php'" class=""><a href="../../vulnerabilities/fi/.?
page=include.php">File Inclusion</a></li>
<li onclick="window.location='../../vulnerabilities/upload/'"
class=""><a href="../../vulnerabilities/upload/">File Upload</a></li>
<li onclick="window.location='../../vulnerabilities/captcha/'"
class=""><a href="../../vulnerabilities/captcha/">Insecure
CAPTCHA</a></li>
```

```
<li onclick="window.location='../../vulnerabilities/sqli/'" class=""><a
href="../../vulnerabilities/sqli/">SQL Injection</a></li>
<li onclick="window.location='../../vulnerabilities/sqli_blind/'"
class=""><a href="../../vulnerabilities/sqli_blind/">SQL Injection
(Blind)</a></li>
<li onclick="window.location='../../vulnerabilities/xss_r/'"
class=""><a href="../../vulnerabilities/xss_r/">XSS
(Reflected)</a></li>
<li onclick="window.location='../../vulnerabilities/xss_s/'"
class=""><a href="../../vulnerabilities/xss_s/">XSS (Stored)</a></li>
</ul><ul class="menuBlocks"><li
onclick="window.location='../../security.php'" class=""><a
href="../../security.php">DVWA Security</a></li>
<li onclick="window.location='../../phpinfo.php'" class=""><a
href="../../phpinfo.php">PHP Info</a></li>
<li onclick="window.location='../../about.php'" class=""><a
href="../../about.php">About</a></li>
</ul><ul class="menuBlocks"><li
onclick="window.location='../../logout.php'" class=""><a
href="../../logout.php">Logout</a></li>
</ul>
                                </div>

                        </div>

                        <div id="main_body">


<div class="body_padded">
        <h1>Vulnerability: Command Injection</h1>

        <div class="vulnerable_code_area">
                <h2>Ping a device</h2>

                <form name="ping" action="#" method="post">
                        <p>
                                Enter an IP address:
                                <input type="text" name="ip" size="30">
                                <input type="submit" name="Submit"
value="Submit">
                        </p>

                </form>
                <pre>root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
pi:x:1000:1000:,,,:/home/pi:/bin/bash
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
ntp:x:102:104::/home/ntp:/bin/false
statd:x:103:65534::/var/lib/nfs:/bin/false
messagebus:x:104:106::/var/run/dbus:/bin/false
```

```
usbmux:x:105:46:usbmux daemon,,,:/home/usbmux:/bin/false
Debian-exim:x:107:110::/var/spool/exim4:/bin/false
mysql:x:108:112:MySQL Server,,,:/nonexistent:/bin/false
postgres:x:109:113:PostgreSQL
administrator,,,:/var/lib/postgresql:/bin/bash
bind:x:110:114::/var/cache/bind:/bin/false
dnsmasq:x:111:65534:dnsmasq,,,:/var/lib/misc:/bin/false
fetchmail:x:106:65534::/var/lib/fetchmail:/bin/false
dovecot:x:112:109:Dovecot mail server,,,:/usr/lib/dovecot:/bin/false
dovenull:x:113:65534:Dovecot login user,,,:/nonexistent:/bin/false
tomcat7:x:114:116::/usr/share/tomcat7:/bin/false
postfix:x:115:117::/var/spool/postfix:/bin/false
vmail:x:5000:5000::/var/mail:/bin/bash
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_req=1 ttl=64 time=0.143 ms
64 bytes from 127.0.0.1: icmp_req=2 ttl=64 time=0.088 ms
64 bytes from 127.0.0.1: icmp_req=3 ttl=64 time=0.129 ms
64 bytes from 127.0.0.1: icmp_req=4 ttl=64 time=0.128 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2997ms
rtt min/avg/max/mdev = 0.088/0.122/0.143/0.020 ms
</pre>
       </div>

       <h2>More Information</h2>
       <ul>
               <li><a
href="http://hiderefer.com/?http://www.scribd.com/doc/2530476/Php-
Endangers-Remote-Code-Execution"
target="_blank">http://www.scribd.com/doc/2530476/Php-Endangers-Remote-
Code-Execution</a></li>
               <li><a
href="http://hiderefer.com/?http://www.ss64.com/bash/"
target="_blank">http://www.ss64.com/bash/</a></li>
               <li><a
href="http://hiderefer.com/?http://www.ss64.com/nt/"
target="_blank">http://www.ss64.com/nt/</a></li>
               <li><a
href="http://hiderefer.com/?https://www.owasp.org/index.php/Command_Inj
ection"
target="_blank">https://www.owasp.org/index.php/Command_Injection</
a></li>
       </ul>
</div>

                               <br /><br />


                       </div>

                       <div class="clear">
                       </div>

                       <div id="system_info">
                               <input type="button" value="View Help"
class="popup_button" onClick="javascript:popUp(
'../../vulnerabilities/view_help.php?id=exec&security=low' )"> <input
type="button" value="View Source" class="popup_button"
onClick="javascript:popUp( '../../vulnerabilities/view_source.php?
id=exec&security=low' )"> <div align="left"><em>Username:</em> admin<br
/><em>Security Level:</em> low<br /><em>PHPIDS:</em> disabled</div>
                       </div>

                       <div id="footer">

                               <p>Damn Vulnerable Web Application (DVWA)
```

```
v1.9</p>

                        </div>

                </div>

        </body>

</html>
```

Recommendation

If at all possible, use library calls rather than external processes to recreate the desired functionality.

Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software.

OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations.
This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise.

For any data that will be used to generate a command to be executed, keep as much of that data out of external control as possible. For example, in web applications, this may require storing the command locally in the session's state instead of sending it out to the client in a hidden form field.

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.

For example, consider using the ESAPI Encoding control or a similar tool, library, or framework. These will help the programmer encode outputs in a manner less prone to error.

If you need to use dynamically-generated query strings or commands in spite of the risk, properly quote arguments and escape any special characters within those arguments. The most conservative approach is to escape or filter all characters that do not pass an extremely strict allow list (such as everything that is not alphanumeric or white space). If some special characters are still needed, such as white space, wrap each argument in quotes after the escaping/filtering step. Be careful of argument injection.

If the program to be executed allows arguments to be specified within an input file or from standard input, then consider using that mode to pass arguments instead of the command line.

If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated.

Some languages offer multiple functions that can be used to invoke

commands. Where possible, identify any function that invokes a command shell using a single string, and replace it with a function that requires individual arguments. These functions typically perform appropriate quoting and filtering of arguments. For example, in C, the system() function accepts a string that contains the entire command to be executed, whereas execl(), execve(), and others require an array of strings, one for each argument. In Windows, CreateProcess() only accepts one command at a time. In Perl, if system() is provided with an array of arguments, then it will quote each of the arguments.

Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

When constructing OS command strings, use stringent allow lists that limit the character set based on the expected value of the parameter in the request. This will indirectly limit the scope of an attack, but this technique is less important than proper output encoding and escaping.

Note that proper output encoding, escaping, and quoting is the most effective solution for preventing OS command injection, although input validation may provide some defense-in-depth. This is because it effectively limits what will appear in output. Input validation will not always prevent OS command injection, especially if you are required to support free-form text fields that could contain arbitrary characters. For example, when invoking a mail program, you might need to allow the subject field to contain otherwise-dangerous inputs like ";" and ">" characters, which would need to be escaped or otherwise handled. In this case, stripping the character might reduce the risk of OS command injection, but it would produce incorrect behavior because the subject field would not be recorded as the user intended. This might seem to be a minor inconvenience, but it could be more important when the program relies on well-structured subject lines in order to pass messages to other components.

Even if you make a mistake in your validation (such as forgetting one out of 100 input fields), appropriate encoding is still likely to protect you from injection-based attacks. As long as it is not done in isolation, input validation is still a useful technique, since it may

	significantly reduce your attack surface, allow you to detect some attacks, and provide other security benefits that proper encoding does not address.
References	CWE: CWE-78 https://cwe.mitre.org/data/definitions/78.htmlhttps://owasp.org/www-community/attacks/Command_Injection WASC:31

4.2.6 Path Traversal

		CVSS 6.5
Title	**Path Traversal**	
Scope	192.168.99.13	
Description	The Path Traversal attack technique allows an attacker access to files, directories, and commands that potentially reside outside the web document root directory. An attacker may manipulate a URL in such a way that the web site will execute or reveal the contents of arbitrary files anywhere on the web server. Any device that exposes an HTTP-based interface is potentially vulnerable to Path Traversal. Most web sites restrict user access to a specific portion of the file-system, typically called the "web document root" or "CGI root" directory. These directories contain the files intended for user access and the executable necessary to drive web application functionality. To access files or execute commands anywhere on the file-system, Path Traversal attacks will utilize the ability of special-characters sequences. The most basic Path Traversal attack uses the "../" special-character sequence to alter the resource location requested in the URL. Although most popular web servers will prevent this technique from escaping the web document root, alternate encodings of the "../" sequence may help bypass the security filters. These method variations include valid and invalid Unicode-encoding ("..%u2216" or "..%c0%af") of the forward slash character, backslash characters ("..\") on Windows-based servers, URL encoded characters "%2e%2e%2f"), and double URL encoding ("..%255c") of the backslash character. Even if the web server properly restricts Path Traversal attempts in the URL path, a web application itself may still be vulnerable due to improper handling of user-supplied input. This is a common problem of web applications that use template mechanisms or load static text from files. In variations of the attack, the original URL parameter value is substituted with the file name of one of the web application's dynamic scripts. Consequently, the results can reveal source code because the file is interpreted as text instead of an executable script. These techniques often employ additional special characters such as the dot (".") to reveal the listing of the current working directory, or "%00" NULL characters in order to bypass rudimentary file extension checks.	
Observation	Target: Hostname(s): Operating System: Tool: External ID:	192.168.99.13 playground.raspwn.org, 192.168.99.13 Linux Zap ZAP-6

Web Vulnerability Findings:
Website: https://192.168.99.13
Path: /dvwa/vulnerabilities/fi/
Parameters: page
Parameter name: page
Query: page=%2Fetc%2Fpasswd
Method: GET

POC

HTTP Request

```
GET https://192.168.99.13/dvwa/vulnerabilities/fi/?page=%2Fetc%2Fpasswd
HTTP/1.1
host: 192.168.99.13
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Referer: https://192.168.99.13/dvwa/vulnerabilities/fi/?page=file1.php
Connection: keep-alive
Cookie: security=low; PHPSESSID=e4b816vebcgoufdmiibn9bcse2
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
```

HTTP Response

```
HTTP/1.1 200 OK
Date: Tue, 06 Sep 2016 13:28:02 GMT
Server: Apache/2.2.22 (Debian)
X-Powered-By: PHP/5.4.36-0+deb7u1
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 5387
Keep-Alive: timeout=5, max=96
Connection: Keep-Alive
Content-Type: text/html;charset=utf-8

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
pi:x:1000:1000:,,,:/home/pi:/bin/bash
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
ntp:x:102:104::/home/ntp:/bin/false
```

```
statd:x:103:65534::/var/lib/nfs:/bin/false
messagebus:x:104:106::/var/run/dbus:/bin/false
usbmux:x:105:46:usbmux daemon,,,:/home/usbmux:/bin/false
Debian-exim:x:107:110::/var/spool/exim4:/bin/false
mysql:x:108:112:MySQL Server,,,:/nonexistent:/bin/false
postgres:x:109:113:PostgreSQL
administrator,,,:/var/lib/postgresql:/bin/bash
bind:x:110:114::/var/cache/bind:/bin/false
dnsmasq:x:111:65534:dnsmasq,,,:/var/lib/misc:/bin/false
fetchmail:x:106:65534::/var/lib/fetchmail:/bin/false
dovecot:x:112:109:Dovecot mail server,,,:/usr/lib/dovecot:/bin/false
dovenull:x:113:65534:Dovecot login user,,,:/nonexistent:/bin/false
tomcat7:x:114:116::/usr/share/tomcat7:/bin/false
postfix:x:115:117::/var/spool/postfix:/bin/false
vmail:x:5000:5000::/var/mail:/bin/bash

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

        <head>
                <meta http-equiv="Content-Type" content="text/html;
charset=UTF-8" />

                <title>Vulnerability: File Inclusion :: Damn Vulnerable
Web Application (DVWA) v1.9</title>

                <link rel="stylesheet" type="text/css"
href="../../dvwa/css/main.css" />

                <link rel="icon" type="\image/ico"
href="../../favicon.ico" />

                <script type="text/javascript"
src="../../dvwa/js/dvwaPage.js"></script>

        </head>

        <body class="home">
                <div id="container">

                        <div id="header">

                                <img src="../../dvwa/images/logo.png"
alt="Damn Vulnerable Web Application" />

                        </div>

                        <div id="main_menu">

                                <div id="main_menu_padded">
                                <ul class="menuBlocks"><li
onclick="window.location='../../.'" class=""><a
href="../../.">Home</a></li>
<li onclick="window.location='../../instructions.php'" class=""><a
href="../../instructions.php">Instructions</a></li>
<li onclick="window.location='../../setup.php'" class=""><a
href="../../setup.php">Setup / Reset DB</a></li>
</ul><ul class="menuBlocks"><li
onclick="window.location='../../vulnerabilities/brute/'" class=""><a
href="../../vulnerabilities/brute/">Brute Force</a></li>
<li onclick="window.location='../../vulnerabilities/exec/'" class=""><a
href="../../vulnerabilities/exec/">Command Injection</a></li>
<li onclick="window.location='../../vulnerabilities/csrf/'" class=""><a
href="../../vulnerabilities/csrf/">CSRF</a></li>
<li onclick="window.location='../../vulnerabilities/fi/.?
```

```
page=include.php'" class="selected"><a
href="../../vulnerabilities/fi/.?page=include.php">File
Inclusion</a></li>
<li onclick="window.location='../../vulnerabilities/upload/'"
class=""><a href="../../vulnerabilities/upload/">File Upload</a></li>
<li onclick="window.location='../../vulnerabilities/captcha/'"
class=""><a href="../../vulnerabilities/captcha/">Insecure
CAPTCHA</a></li>
<li onclick="window.location='../../vulnerabilities/sqli/'" class=""><a
href="../../vulnerabilities/sqli/">SQL Injection</a></li>
<li onclick="window.location='../../vulnerabilities/sqli_blind/'"
class=""><a href="../../vulnerabilities/sqli_blind/">SQL Injection
(Blind)</a></li>
<li onclick="window.location='../../vulnerabilities/xss_r/'"
class=""><a href="../../vulnerabilities/xss_r/">XSS
(Reflected)</a></li>
<li onclick="window.location='../../vulnerabilities/xss_s/'"
class=""><a href="../../vulnerabilities/xss_s/">XSS (Stored)</a></li>
</ul><ul class="menuBlocks"><li
onclick="window.location='../../security.php'" class=""><a
href="../../security.php">DVWA Security</a></li>
<li onclick="window.location='../../phpinfo.php'" class=""><a
href="../../phpinfo.php">PHP Info</a></li>
<li onclick="window.location='../../about.php'" class=""><a
href="../../about.php">About</a></li>
</ul><ul class="menuBlocks"><li
onclick="window.location='../../logout.php'" class=""><a
href="../../logout.php">Logout</a></li>
</ul>
                                </div>

                        </div>

                        <div id="main_body">


                                <br /><br />


                        </div>

                        <div class="clear">
                        </div>

                        <div id="system_info">
                                <input type="button" value="View Help"
class="popup_button" onClick="javascript:popUp(
'../../vulnerabilities/view_help.php?id=fi&security=low' )"> <input
type="button" value="View Source" class="popup_button"
onClick="javascript:popUp( '../../vulnerabilities/view_source.php?
id=fi&security=low' )"> <div align="left"><em>Username:</em>
admin<br /><em>Security Level:</em> low<br /><em>PHPIDS:</em>
disabled</div>
                        </div>

                        <div id="footer">

                                <p>Damn Vulnerable Web Application (DVWA)
v1.9</p>

                        </div>

                </div>

        </body>
```

```
</html>
```

Recommendation

Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

For filenames, use stringent allow lists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses, and exclude directory separators such as "/". Use an allow list of allowable file extensions.

Warning: if you attempt to cleanse your data, then do so that the end result is not in the form that can be dangerous. A sanitizing mechanism can remove characters such as '.' and ';' which may be required for some exploits. An attacker can try to fool the sanitizing mechanism into "cleaning" data into a dangerous form. Suppose the attacker injects a '.' inside a filename (e.g. "sensi.tiveFile") and the sanitizing mechanism removes the character resulting in the valid filename, "sensitiveFile". If the input data are now assumed to be safe, then the file may be compromised.

Inputs should be decoded and canonicalized to the application's current internal representation before being validated. Make sure that your application does not decode the same input twice. Such errors could be used to bypass allow list schemes by introducing dangerous inputs after they have been checked.

Use a built-in path canonicalization function (such as realpath() in C) that produces the canonical version of the pathname, which effectively removes ".." sequences and symbolic links.

Run your code using the lowest privileges that are required to accomplish the necessary tasks. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations.

When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs.

Run your code in a "jail" or similar sandbox environment that

	enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software. OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise.
References	CWE: CWE-22 https://owasp.org/www-community/attacks/Path_Traversalhttps://cwe.mitre.org/data/definitions/22.html WASC:33

4.2.7 Cleartext Transmission of Sensitive Information via HTTP

		CVSS 5.4
Title	**Cleartext Transmission of Sensitive Information via HTTP**	
Scope	192.168.99.1	
Description	The host / application transmits sensitive information (username, passwords) in cleartext via HTTP.	
Observation	Target: 192.168.99.1 Hostname(s): ns1.playground.raspwn.org, _gateway Operating System: Linux Tool: Openvas External ID: OPENVAS-1.3.6.1.4.1.25623.1.0.108440	
POC		
Recommendation	Enforce the transmission of sensitive data via an encrypted SSL/TLS connection. Additionally make sure the host / application is redirecting all users to the secured SSL/TLS connection before allowing to input sensitive data into the mentioned functions.	
References	SEVERITY NUMBER: 4.8 THREAT: Medium	

4.2.8 phpinfo() Output Reporting (HTTP)

		CVSS 5.3
Title	**phpinfo() Output Reporting (HTTP)**	
Scope	192.168.99.13	
Description	Reporting of files containing the output of the phpinfo() PHP function previously detected via HTTP. Many PHP installation tutorials instruct the user to create a file called phpinfo.php or similar containing the phpinfo() statement. Such a file is often left back in the webserver directory. http://playground.raspwn.org/mutillidae/phpinfo.php	
Observation	Target: 192.168.99.13 Hostname(s): playground.raspwn.org, 192.168.99.13 Operating System: Linux Tool: Openvas External ID: OPENVAS-1.3.6.1.4.1.25623.1.0.11229	
POC		
Recommendation	Delete the listed files or restrict access to them.	
References	SEVERITY NUMBER: 5.3 THREAT: Medium	

4.2.9 SSL/TLS: Certificate Expired

		CVSS 5.3
Title	**SSL/TLS: Certificate Expired**	
Scope	192.168.99.13	
Description	The remote server's SSL/TLS certificate has already expired. This script checks expiry dates of certificates associated with SSL/TLS-enabled services on the target and reports whether any have already expired.	
Observation	Target: 192.168.99.13 Hostname(s): playground.raspwn.org, 192.168.99.13 Operating System: Linux Tool: Openvas External ID: OPENVAS-1.3.6.1.4.1.25623.1.0.103955	
POC		
Recommendation	Replace the SSL/TLS certificate by a new one.	
References	cpe:/a:ietf:transport_layer_security SEVERITY NUMBER: 5.0 THREAT: Medium	

5. ANNEXES

5.1 FIGURES TABLE

Figure 1 - name.....0